FILLING STATION MANAGEMENT SYSTEM



PRESENTED BY :

Dewan Towhidur Rahman — 24201205
Saad Bin Harun — 24201212
Maisha Fahmida Islam — 24101228

PRESENTED TO :

Alif Ruslan

Lecturer, Department of Computer Science and Engineering
University Of Asia Pacific

STRATEGIC CONSULTING

TEAM



CTO & CO-FOUNDER

Maisha Fahmida Islam



HEAD OF MARKETING

Dewan Towhidur Rahman



ASSISTANT

MD. Saad Bin Harun

INTRODUCTION & PROBLEM STATEMENT



1.

CUSTOMER PERSPECTIVE

- **Fuel Price Uncertainty:** Customers cannot easily predict fuel price changes or secure a favorable rate.
- **Limited Accessibility:** Customers may need to visit a specific fuel station, limiting their choices.
- **Long Waiting Time:** Traffic congestion and queues at fuel stations waste customers' time.
- **Lack of Information:** Customers have limited access to real-time fuel prices, availability, and nearby stations.



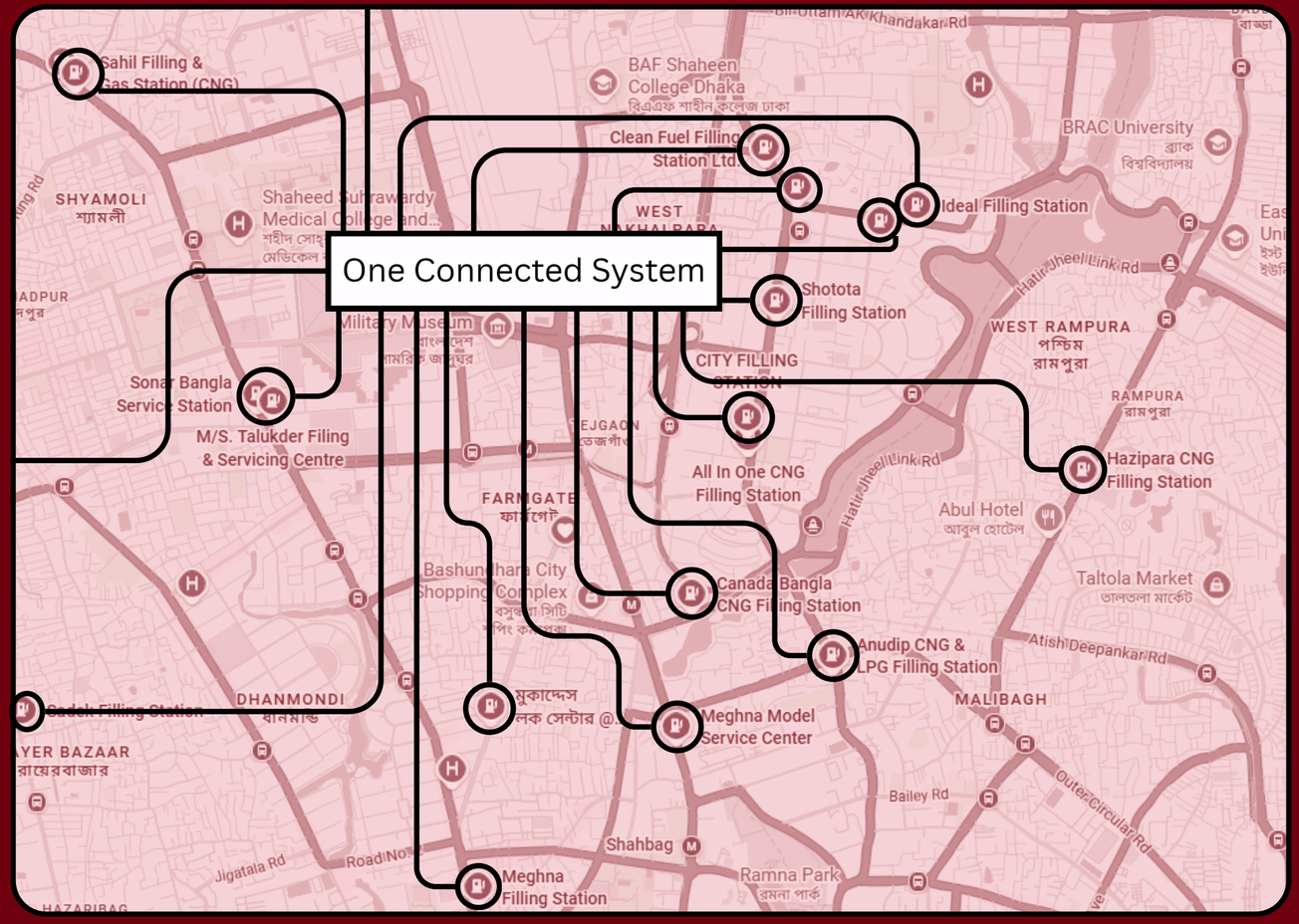
2.

FUEL STATION OWNER PERSPECTIVE

- **Manual Operations:** Manual sales recording and stock tracking can lead to errors and inefficiencies.
- **Inventory Management:** Difficulty monitoring fuel stock levels, consumption, and replenishment.
- **Disconnected Data:** Lack of a centralized system to manage customer, sales, and inventory data across stations.
- **Lack of Analytics:** Limited access to useful insights into sales trends, customer demand, and business performance.

WHAT DID WE PROPOSE

CONNECTING STATION OPERATIONS, CUSTOMERS, SALES, PAYMENTS, AND INVENTORY



OBJECTIVES & SCOPE



REAL-TIME INFORMATION:

Provide updated fuel prices and availability at nearby fuel stations.



IMPROVED ACCESSIBILITY:

Allow customers to find and choose suitable fuel stations through a single platform.



REDUCED WAITING TIME:

Help customers identify stations with available fuel and potentially shorter queues.



PRICE TRANSPARENCY:

Help customers compare fuel prices and make informed decisions.



CENTRALIZED MANAGEMENT:

Manage sales, fuel inventory, and station information through a unified system.



AUTOMATED OPERATIONS:

Reduce manual errors in sales records, stock tracking, and daily operations.



INVENTORY MONITORING:

Track fuel stock levels and receive alerts when replenishment is needed.



DATA-DRIVEN DECISIONS:

Generate reports and analyze sales trends, fuel demand, and business performance.



STATION NETWORKING:

Connect multiple fuel stations to share updated information and improve coordination.

SYSTEM DESIGN

ENTITY SETS

1. Fuel
- Fuel_Id(Primary Key)
- Fuel_Name
- Price_per_Leter
- Status

2. Storage_Tank
- Tank_Id(Primary Key)
- Fuel_Id(Foreign Key)
- Capacity
- Stock
- Location
- Pump_Id

3. Vehicle
- Vehicle_Id(Primary Key)
- Vehicle_type
- Fuel_Id(Foreign Key)
- Customer_Id(Foreign Key)

4. Customer
- Customer_Id(Primary Key)
- Customer_Name
- Customer_Phone

5. Sales_Transaction
- Sale_Id(Primary Key)
- Customer_Id(Foreign Key)
- Sale_Date
- Quantity
- Total_amount

6. Payment
- Payment_Id(Primary Key)
- Sale_Id(Foreign Key)
- Payment_Type
- Status

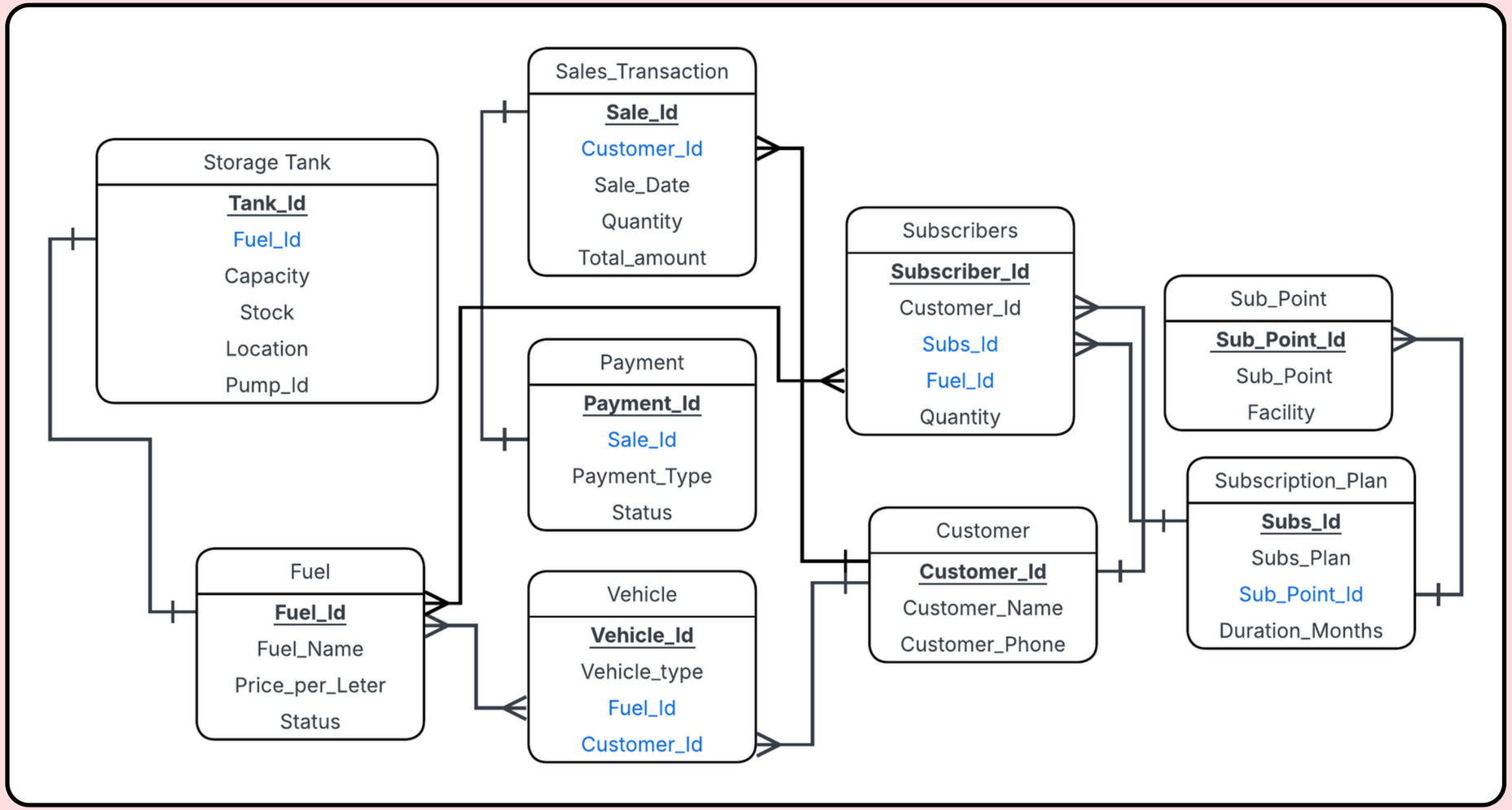
7. Subscribers
- Subscriber_Id(Primary Key)
- Customer_Id(Foreign Key)
- Subs_Id(Foreign Key)
- Fuel_Id(Foreign Key)
- Quantity

8. Subscription_Plan
- Subs_Id(Primary Key)
- Subs_Plan
- Sub_Point_Id(Foreign Key)
- Duration_Months

9. Sub_Point
- Sub_Point_Id(Primary Key)
- Sub_Point

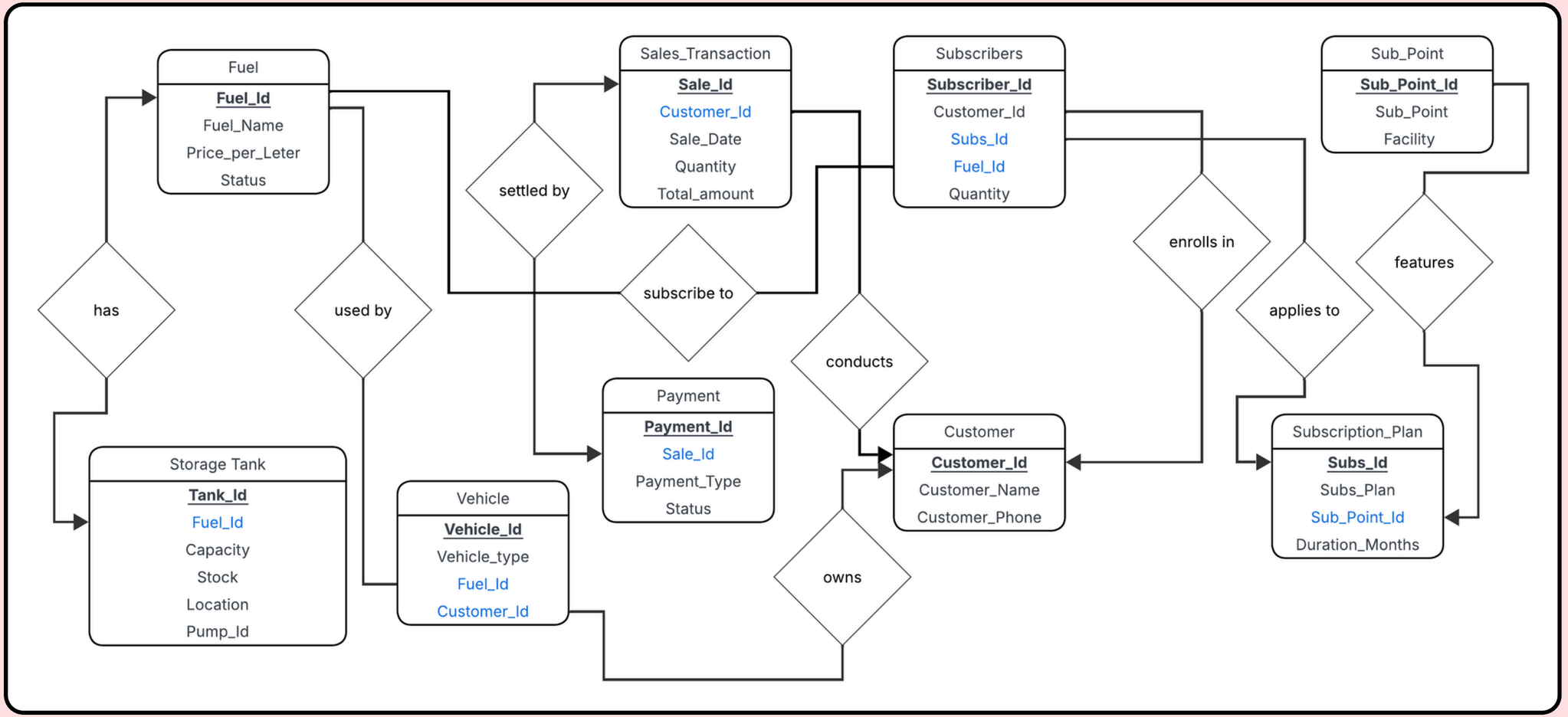
SYSTEM DESIGN

SCHEMA DIAGRAM



SYSTEM DESIGN

ER DIAGRAM



SYSTEM DESIGN

RELATIONS

- **Customer – Vehicle (owns): 1:M** — A customer can own multiple vehicles; each vehicle belongs to one customer.
- **Customer – Sales_Transaction (conducts): 1:M** — A customer can make multiple transactions; each transaction belongs to one customer.
- **Customer – Subscribers (enrolls in): 1:M** — A customer can have multiple subscription records; each record belongs to one customer.
- **Subscription_Plan – Subscribers (applies to): 1:M** — A plan can apply to multiple subscribers; each subscriber uses one plan.
- **Sales_Transaction – Payment (settled by): 1:1** — Each transaction has one payment, and each payment settles one transaction.
- **Fuel – Storage_Tank (has): 1:1** — Each fuel type is linked to one storage tank, and vice versa.
- **Sub_Point – Subscription_Plan (features): M:1** — Multiple sub-points feature one subscription plan.
- **Fuel – Vehicle (used by): M:N** — A fuel type is used by many vehicles, and vehicles can use multiple fuels.
- **Fuel – Subscribers (subscribe to): M:N** — A fuel type can have many subscribers, and subscribers can link to multiple fuels.

DATABASE DESIGN

```
CREATE TABLE FUEL (
  FUEL_ID INTEGER PRIMARY KEY,
  FUEL_NAME TEXT NOT NULL,
  PRICE_PER_LITER DECIMAL(6,2) NOT NULL,
  STATUS TEXT NOT NULL
);
```

Fuel_Id	Fuel_Name	Price_per_liter (৳)	Status
1	Octen	145.00	available
2	Diesel	115.00	available
3	LPG	70.40	available
4	Lubricant	450.00	not available
5	Petrol / Gasoline	140.00	not available

```
CREATE TABLE STORAGETANK (
  TANK_ID INTEGER PRIMARY KEY,
  FUEL_ID INTEGER,
  CAPACITY INT,
  STOCK INT,
  LOCATION VARCHAR(50),
  PUMP_ID INT,
  FOREIGN KEY (FUEL_ID) REFERENCES FUEL(FUEL_ID)
);
```

Tank_Id	Fuel_Id	Capacity	Stock	Location	Pump_Id
1	1	10000	6450	Uttara Sector 3	201
2	2	15000	11200	Mirpur 12	202
3	3	8000	4300	Gulshan Avenue	203
4	4	5000	1500	Banani Road 11	204
5	5	12000	8900	Dhanmondi 27	205
6	1	10000	7800	Tejgaon Industrial Area	206
...	...	...	...	...	...
30	5	10000	6800	Paltan	230

DATABASE DESIGN

```
CREATE TABLE CUSTOMER (
  CUSTOMER_ID INTEGER PRIMARY KEY,
  CUSTOMER_NAME VARCHAR(20) NOT NULL,
  CUSTOMER_PHONE_NO VARCHAR(15)
);
```

Customer_Id	Customer_Name	Customer_phone_no
1	Tanvir Ahmed	01712-458921
2	Fariha Tabassum	01834-902314
3	Shahriar Hossain	01925-671403
...	...	...
5	Asif Akbar	01722-556677

```
CREATE TABLE VEHICLE (
  VEHICLE_ID INTEGER PRIMARY KEY,
  VEHICLE_TYPE VARCHAR(30) NOT NULL,
  FUEL_ID INTEGER,
  CUSTOMER_ID INTEGER,
  FOREIGN KEY (FUEL_ID) REFERENCES FUEL(FUEL_ID),
  FOREIGN KEY (CUSTOMER_ID) REFERENCES
 CUSTOMER(CUSTOMER_ID)
);
```

Vehicle_Id	Vehicle_type	Fuel_Id	Customer_Id
101	Bike	1	1
102	Car	2	1
103	Bus	3	1
104	Truck	2	2
105	Microbus	5	2
...	...	...	...
130	Postal Van	1	26

DATABASE DESIGN

```
CREATE TABLE SALES_TRANSACTION (
  SALE_ID INTEGER PRIMARY KEY,
  CUSTOMER_ID INTEGER NOT NULL,
  SALE_DATE DATE NOT NULL,
  QUANTITY DECIMAL(20,2),
  TOTAL_AMOUNT DECIMAL(20,2),
  FOREIGN KEY (CUSTOMER_ID)
    REFERENCES CUSTOMER(CUSTOMER_ID)
);
```

Sale_Id	Customer_Id	Sale_date	Quantity	Total_amount
1001	1	2026-01-10	45.00	6525.00
1002	2	2026-01-12	32.50	3737.50
1003	3	2026-01-15	20.00	1408.00
1004	4	2026-01-18	60.00	6900.00
1005	5	2026-01-20	15.50	2247.50
...	...	...	...	...
1030	30	2026-04-05	40.00	4600.00

```
CREATE TABLE PAYMENT (
  PAYMENT_ID INTEGER PRIMARY KEY,
  SALE_ID INTEGER NOT NULL,
  PAYMENT_TYPE VARCHAR(20) NOT NULL,
  STATUS VARCHAR(20) NOT NULL,
  FOREIGN KEY (SALE_ID)
    REFERENCES SALES_TRANSACTION(SALE_ID)
);
```

Payment_Id	Sale_Id	Payment_Type	Status
5001	1001	Pre-Paid	Paid
5002	1002	Post-Paid	Paid
5003	1003	Pre-Paid	Paid
5004	1004	Post-Paid	Paid
5005	1005	Pre-Paid	Pending
...	...	...	...
5030	1030	Post-Paid	Paid

DATABASE DESIGN

```
CREATE TABLE SUB_POINT (
  SUB_POINT_ID INT PRIMARY KEY AUTO_INCREMENT,
  SUB_POINT INT NOT NULL,
  FACILITY VARCHAR(50)
);
```

Sub_Point_Id	Sub_Point	Facility
1	100	Car Wash
2	200	Free Oil change
3	300	Discount

```
CREATE TABLE SUBSCRIPTION_PLAN (
  SUBS_ID INT PRIMARY KEY AUTO_INCREMENT,
  SUBS_PLAN VARCHAR(20) NOT NULL,
  SUB_POINT_ID INT,
  DURATION_MONTHS INT NOT NULL,
  FOREIGN KEY (SUB_POINT_ID)
    REFERENCES SUB_POINT(SUB_POINT_ID)
);
```

Subs_Id	Subs_Plan	Sub_Point_Id	Duration_Months
1	3 month	1	3
2	2 month	2	2
3	1 month	3	1

DATABASE DESIGN

```
CREATE TABLE SUBSCRIBERS (
  SUBSCRIBER_ID INT PRIMARY KEY AUTO_INCREMENT,
  CUSTOMER_ID INT NOT NULL,
  SUBS_ID INT NOT NULL,
  FUEL_ID INT NOT NULL,
  QUANTITY DECIMAL(10,2) NOT NULL,

  FOREIGN KEY (CUSTOMER_ID)
    REFERENCES CUSTOMER(CUSTOMER_ID),

  FOREIGN KEY (SUBS_ID)
    REFERENCES SUBSCRIPTION_PLAN(SUBS_ID),

  FOREIGN KEY (FUEL_ID)
    REFERENCES FUEL(FUEL_ID)


);
```

Subscriber_Id	Customer_Id	Subs_Id	Fuel_Id	Quantity
1	1	1	1	50.00
2	1	2	1	30.00
3	1	3	2	100.00
4	2	1	2	120.00
5	2	2	5	80.00
...	...	...	...	...
30	26	1	1	35.00

SQL QURIES

Easy way to see every available fuel's with their price

```
SELECT * FROM Fuel
WHERE Status = 'available';
```

```
+---------+-----------+-----------------+-----------+
| Fuel_Id | Fuel_Name | Price_per_liter | Status    |
+---------+-----------+-----------------+-----------+
|       1 | Octen     |          145.00 | available |
|       2 | Diesel    |          115.00 | available |
|       3 | LPG       |           70.40 | available |
+---------+-----------+-----------------+-----------+
```

Sort Fuel Prices

```
SELECT * FROM Fuel
ORDER BY Price_per_liter ASC;
```

```
+---------+------------------+----------------+--------------+
| Fuel_Id | Fuel_Name        | Price_per_liter | Status       |
+---------+------------------+----------------+--------------+
|       3 | LPG              |           70.40 | available    |
|       2 | Diesel           |          115.00 | available    |
|       5 | Petrol / Gasoline |          140.00 | not available |
|       1 | Octen            |          145.00 | available    |
|       4 | Lubricant        |          450.00 | not available |
+---------+------------------+----------------+--------------+
```

Search Customer by Name

```
SELECT * FROM Customer
WHERE Customer_Name LIKE 'T%';
```

```
+-------------+---------------+-------------------+
| Customer_Id | Customer_Name | Customer_phone_no |
+-------------+---------------+-------------------+
|           1 | Tanvir Ahmed  | 01712-458921      |
|          18 | Tania Sultana | 01955-778899      |
+-------------+---------------+-------------------+
```

SQL QURIES

Find Pre-Paid or Pending Payments

```
SELECT * FROM Payment
WHERE Payment_Type = 'Pre-Paid'
OR Status = 'Pending';
```

Payment_Id	Sale_Id	Payment_Type	Status
5001	1001	Pre-Paid	Paid
5003	1003	Pre-Paid	Paid
5005	1005	Pre-Paid	Pending
5007	1007	Pre-Paid	Paid
5009	1009	Pre-Paid	Paid
5010	1010	Post-Paid	Pending
5011	1011	Pre-Paid	Paid
5013	1013	Pre-Paid	Paid
5015	1015	Pre-Paid	Pending
5017	1017	Pre-Paid	Paid
5018	1018	Pre-Paid	Paid
5019	1019	Pre-Paid	Paid
5020	1020	Post-Paid	Pending
5021	1021	Pre-Paid	Paid
5023	1023	Pre-Paid	Paid
5025	1025	Pre-Paid	Paid
5026	1026	Post-Paid	Pending
5027	1027	Pre-Paid	Paid
5029	1029	Pre-Paid	Paid

SQL QURIES

Count Total Customers and Calculate Total Sales

```
SELECT
    C.Total_Customers,
    S.Total_Sales
FROM
    (SELECT COUNT(*) AS Total_Customers
     FROM Customer) AS C
CROSS JOIN
    (SELECT SUM(Total_amount) AS Total_Sales
     FROM Sales_transaction) AS S;
```

```
+-----------------+-------------+
| Total_Customers | Total_Sales |
+-----------------+-------------+
|              30 |   149745.40 |
+-----------------+-------------+
```

Find Fuel Used by More Than 5 Vehicles

```
SELECT F.Fuel_Id, F.Fuel_Name,
       COUNT(V.Vehicle_Id) AS Total_Vehicles
FROM Vehicle V
INNER JOIN Fuel F
ON V.Fuel_Id = F.Fuel_Id
GROUP BY F.Fuel_Id, F.Fuel_Name
HAVING COUNT(V.Vehicle_Id) > 5;
```

```
+---------+-----------+----------------+
| Fuel_Id | Fuel_Name | Total_Vehicles |
+---------+-----------+----------------+
|       1 | Octen     |              9 |
|       2 | Diesel    |             13 |
+---------+-----------+----------------+
```

SQL QURIES

Find Fuels Above Average Price

```
SELECT Fuel_Name, Price_per_liter
FROM Fuel
WHERE Price_per_liter >
(
    SELECT AVG(Price_per_liter)
    FROM Fuel
);
```

```
+-----------+-----------------+
| Fuel_Name | Price_per_liter |
+-----------+-----------------+
| Lubricant |          450.00 |
+-----------+-----------------+
```

Categorize Fuel Prices

```
SELECT Fuel_Name, Price_per_liter,
CASE
    WHEN Price_per_liter < 100 THEN 'Cheap'
    WHEN Price_per_liter <= 200 THEN 'Medium'
    ELSE 'Expensive'
END AS Price_Category
FROM Fuel;
```

```
+-------------------+-----------------+----------------+
| Fuel_Name         | Price_per_liter | Price_Category |
+-------------------+-----------------+----------------+
| Octen             |          145.00 | Medium         |
| Diesel            |          115.00 | Medium         |
| LPG               |           70.40 | Cheap          |
| Lubricant         |          450.00 | Expensive      |
| Petrol / Gasoline |          140.00 | Medium         |
+-------------------+-----------------+----------------+
```

SQL QURIES

Display the Customer's name , sale's amount and payment status

```
SELECT Customer.Customer_Name,
       Sales_transaction.Total_amount,
       Payment.Payment_Type,
       Payment.Status
FROM Customer
INNER JOIN Sales_transaction
ON Customer.Customer_Id = Sales_transaction.Customer_Id
INNER JOIN Payment
ON Sales_transaction.Sale_Id = Payment.Sale_Id;
```

Customer_Name	Total_amount	Payment_Type	Status
Tanvir Ahmed	6525.00	Pre-Paid	Paid
Fariha Tabassum	3737.50	Post-Paid	Paid
Shahriar Hossain	1408.00	Pre-Paid	Paid
Nusrat Jahan	6900.00	Post-Paid	Paid
Rakib Hasan	2247.50	Pre-Paid	Pending
Sadia Afrin	4600.00	Post-Paid	Paid
Imtiaz Mahmud	2875.00	Pre-Paid	Paid
Lamia Islam	8625.00	Post-Paid	Paid
Zubayer Rahman	4350.00	Pre-Paid	Paid
Farzana Yasmin	3520.00	Post-Paid	Pending
Mahbub Alam	5075.00	Pre-Paid	Paid
Anika Tasnim	10350.00	Post-Paid	Paid
Shariful Islam	1740.00	Pre-Paid	Paid
Priya Sarker	6325.00	Post-Paid	Paid
Sabbir Hossain	9200.00	Pre-Paid	Pending
Mumu Akter	3190.00	Post-Paid	Paid
Rashed Khan	5175.00	Pre-Paid	Paid
Tania Sultana	1302.40	Pre-Paid	Paid
Al-Amin Mia	7475.00	Pre-Paid	Paid
Bristy Roy	5510.00	Post-Paid	Pending
Fahim Faisal	4830.00	Pre-Paid	Paid
Nadia Chowdhury	3220.00	Post-Paid	Paid
Sumon Karmakar	5750.00	Pre-Paid	Paid
Mehedi Hasan	4620.00	Post-Paid	Paid
Nazmul Haque	8050.00	Pre-Paid	Paid
Sabrina Yesmin	1450.00	Post-Paid	Pending
Juel Rana	5520.00	Pre-Paid	Paid
Moniruzzaman	2875.00	Post-Paid	Paid
Rumana Khatun	8700.00	Pre-Paid	Paid
Asif Akbar	4600.00	Post-Paid	Paid

SQL QURIES

Display Customer's whose total sale's amount is greater then 5000

```
SELECT Customer.Customer_Name,
       SUM(Sales_transaction.Total_amount) AS Total_Sales
FROM Customer
INNER JOIN Sales_transaction
ON Customer.Customer_Id = Sales_transaction.Customer_Id
GROUP BY Customer.Customer_Id, Customer.Customer_Name
HAVING SUM(Sales_transaction.Total_amount) > 5000
ORDER BY Total_Sales DESC;
```

Customer_Name	Total_Sales
Anika Tasnim	10350.00
Sabbir Hossain	9200.00
Rumana Khatun	8700.00
Lamia Islam	8625.00
Nazmul Haque	8050.00
Al-Amin Mia	7475.00
Nusrat Jahan	6900.00
Tanvir Ahmed	6525.00
Priya Sarker	6325.00
Sumon Karmakar	5750.00
Juel Rana	5520.00
Bristy Roy	5510.00
Rashed Khan	5175.00
Mahbub Alam	5075.00

CONCLUSION

- FASTER AND MORE ACCURATE SALES MANAGEMENT
- BETTER FUEL INVENTORY TRACKING
- CENTRALIZED CUSTORMER AND STATION RECORDS
- EASIER PAYMENT AND TRANSACTION MANAGEMENT
- REDUCED MANUAL WORK

THANK YOU

